

1 Understanding object/relational persistence

This chapter covers

- Persisting with SQL databases in Java applications
- Analyzing the object/relational paradigm mismatch
- Introducing ORM, JPA, Hibernate, and Spring Data

This book is about JPA, Hibernate, and Spring Data; our focus is on using Hibernate as a provider of the Jakarta Persistence API (formerly Java Persistence API) and Spring Data as a Spring-based programming model for data access. We'll cover basic and advanced features and describe some ways to develop new applications using the Java Persistence API. Often these recommendations aren't specific to Hibernate or Spring Data. Sometimes they're our own ideas about the *best* ways to do things when working with persistent data, explained in the context of Hibernate and Spring Data.

The choice of approach to managing persistent data may be a key design decision in many software projects. Persistence has always been a hot topic of debate in the Java community. Is persistence a problem that has already been solved by SQL and extensions such as stored procedures, or is it a more pervasive problem that must be addressed by special Java frameworks? Should we hand-code even the most primitive CRUD (create, read, update, delete) operations in SQL and JDBC, or should this work be handed to an intermediary layer? How can we achieve portability if every database management system has its own SQL dialect? Should we abandon SQL completely and adopt a different database technology, such as object database systems or NoSQL systems? The debate may never end, but a solution called *object/relational mapping* (ORM) now has wide acceptance. This is due in large part to Hibernate, an open source ORM service implementation, and Spring Data, an umbrella project from the Spring family whose purpose is to unify and facilitate access to different kinds of persistence stores, including relational database systems and NoSQL databases.

Before we can get started with Hibernate and Spring Data, however, you need to understand the core problems of object persistence and ORM. This chapter explains why you need tools like Hibernate and Spring Data and specifications such as the *Jakarta Persistence API* (JPA).

First we'll define persistent data management in the context of software applications and discuss the relationships between SQL, JDBC, and Java, the underlying technologies and standards that Hibernate and Spring Data build on. We'll then discuss the so-called *object/relational paradigm mismatch* and the generic problems we encounter in object-oriented software development with SQL databases. These problems make it clear that we need tools and patterns to minimize the time we have to spend on persistence-related code in our applications.

The best way to learn Hibernate and Spring Data isn't necessarily linear. We understand that you may want to try Hibernate or Spring Data right away. If this is how you'd like to proceed, skip to the next chapter and set up a project with the "Hello World" example. We recommend that you return here at some point as you go through this book; that way, you'll be prepared and have the background concepts you need for the rest of the material.

1.1 What is persistence?

Most applications require persistent data. Persistence is one of the fundamental concepts in application development. If an information system didn't preserve data when it was powered off, the system would be of little practical use. *Object persistence* means individual objects can outlive the application process; they can be saved to a data store and be re-created at a later point in time. When we talk about persistence in Java, we're generally talking about mapping and storing object instances in a database using SQL.

We'll start by taking a brief look at persistence and how it's used in Java. Armed with this information, we'll continue our discussion of persistence and look at how it's implemented in object-oriented applications.

1.1.1 Relational databases

You, like most other software engineers, have probably worked with SQL and relational databases; many of us handle such systems every day. Relational database management systems have SQL-based application programming interfaces, so we call today's relational database products

SQL *database management systems* (DBMS) or, when we're talking about particular systems, SQL *databases*.

Relational technology is a well-known technology, and this alone is sufficient reason for many organizations to choose it. Relational databases are also an incredibly flexible and robust approach to data management. Due to the well-researched theoretical foundation of the relational data model, relational databases can guarantee and protect the integrity of stored data, along with having other desirable characteristics. You may be familiar with E.F. Codd's five-decade-old introduction of the relational model, "A Relational Model of Data for Large Shared Data Banks" (Codd, 1970). A more recent compendium worth reading, with a focus on SQL, is C.J. Date's *SQL and Relational Theory* (Date, 2015).

Relational DBMSs aren't specific to Java, nor is an SQL database specific to a particular application. This important principle is known as *data independence*. In other words, *data usually lives longer than an application does*. Relational technology provides a way of sharing data among different applications, or among different parts of the same overall system (a data entry application and a reporting application, for example). Relational technology is a common denominator of many disparate systems and technology platforms. Hence, the relational data model is often the foundation for the enterprise-wide representation of business entities.

Before we go into more detail about the practical aspects of SQL databases, we need to mention an important concern: although marketed as relational, a database system providing only an SQL data language interface isn't really relational, and in many ways it isn't even close to the original concept. Naturally, this has led to confusion. SQL practitioners blame the relational data model for shortcomings in the SQL language, and relational data management experts blame the SQL standard for being a weak implementation of the relational model and ideals. We'll highlight some significant aspects of this problem throughout this book, but generally we'll focus on the practical aspects. If you're interested in more background material, we highly recommend *Fundamentals of Database Systems* by Ramez Elmasri and Shamkant B. Navathe (Elmasri, 2016) for the theory and concepts of relational database systems.

1.1.2 Understanding SQL

To use JPA, Hibernate, and Spring Data effectively, you must start with a solid understanding of the relational model and SQL. You'll need to understand the relational model and the information model and topics such

as normalization to guarantee the integrity of your data, and you'll need to use your knowledge of SQL to tune the performance of your application—these are all prerequisites for reading this book. Hibernate and Spring Data simplify many repetitive coding tasks, but your knowledge of persistence technology must extend beyond the frameworks themselves if you want to take advantage of the full power of modern SQL databases. To dig deeper, consult the sources in the references list at the end of this book.

You've probably used SQL for many years and are familiar with the basic operations and statements written in this language. Still, we know from our own experience that SQL is sometimes hard to remember, and some terms vary in usage.

You should be comfortable with them, so let's briefly review some of the SQL terms we'll use in this book. SQL is used as a *data definition language* (DDL), with syntax for *creating*, *altering*, and *dropping* artifacts such as tables and constraints in the catalog of the DBMS. When this *schema* is ready, you can use SQL as a *data manipulation language* (DML) to perform operations on data, including *insertions*, *updates*, and *deletions*. You can retrieve data by executing *data query language* (DQL) statements with *restrictions*, *projections*, and *Cartesian products*. For efficient reporting, you can use SQL to *join*, *aggregate*, and *group* data as necessary. You can even nest SQL statements inside each other—a technique that uses *subselects*. When your business requirements change, you'll have to modify the database schema again with DDL statements after data has been stored; this is known as *schema evolution*. You may also use SQL as a *data control language* (DCL) to *grant and revoke* access to the database or parts of it.

If you're an SQL veteran and you want to know more about optimization and how SQL is executed, get a copy of the excellent book *SQL Tuning* by Dan Tow (Tow, 2003). For a look at the practical side of SQL through the lens of how not to use SQL, *SQL Antipatterns: Avoiding the Pitfalls of Database Programming*, by Bill Karwin (Karwin, 2010) is a good resource.

Although the SQL database is one part of ORM, the other part, of course, consists of the data in your Java application that needs to be persisted to and loaded from the database.

1.1.3 Using SQL in Java

When you work with an SQL database in a Java application, you issue SQL statements to the database via the Java Database Connectivity (JDBC) API. Whether the SQL was written by hand and embedded in the Java

code or generated on the fly by Java code, you use the JDBC API to bind arguments when preparing query parameters, executing a query, scrolling through query results, retrieving values from a result set, and so on. These are low-level data access tasks; as application engineers, we're more interested in the business problem that requires this data access. What we'd really like to write is code that saves and retrieves instances of our classes, relieving us of this low-level labor.

Because these data access tasks are often so tedious, we have to ask: are the relational data model and (especially) SQL the right choices for persistence in object-oriented applications? We can answer this question unequivocally: yes! There are many reasons why SQL databases dominate the computing industry—relational database management systems are the only proven generic data management technology, and they're almost always a *requirement* in Java projects.

Note that we aren't claiming that relational technology is *always* the best solution. Many data management requirements warrant a completely different approach. For example, internet-scale distributed systems (web search engines, content distribution networks, peer-to-peer sharing, instant messaging) have to deal with exceptional transaction volumes. Many of these systems don't require that after a data update completes, all processes see the same updated data (strong transactional consistency). Users might be happy with weak consistency; after an update, there might be a window of inconsistency before all processes see the updated data. In contrast, some scientific applications work with enormous but very specialized datasets. Such systems and their unique challenges typically require equally unique and often custom-made persistence solutions. Generic data management tools such as ACID-compliant transactional SQL databases, JDBC, Hibernate, and Spring Data would play only a minor role for these types of systems.

Relational systems at internet scale

To understand why relational systems, and the data-integrity guarantees associated with them, are difficult to scale, we recommend that you first familiarize yourself with the *CAP theorem*. According to this rule, a distributed system cannot be *consistent*, *available*, and *tolerant against partition failures* all at the same time.

A system may guarantee that all nodes will see the same data at the same time and that data read and write requests are always answered. But when a part of the system fails due to a host, network, or data center problem, you must either give up strong consistency or 100% availability.

In practice, this means you need a strategy that detects partition failures and restores either consistency or availability to a certain degree (for example, by making some part of the system temporarily unavailable so data synchronization can occur in the background). Often, the data, the user, or the operation will determine whether strong consistency is necessary.

In this book, we'll consider the problems of data storage and sharing in the context of an object-oriented application that uses a *domain model*. Instead of directly working with the rows and columns of a `java.sql.ResultSet`, the business logic of the application will interact with the application-specific object-oriented domain model. If the SQL database schema of an online auction system has `ITEM` and `BID` tables, for example, the Java application defines corresponding `Item` and `Bid` classes. Instead of reading and writing the value of a particular row and column with the `ResultSet` API, the application loads and stores instances of `Item` and `Bid` classes.

At runtime, the application therefore operates with instances of these classes. Each instance of a `Bid` has a reference to an auction `Item`, and each `Item` may have a collection of references to `Bid` instances. The business logic isn't executed in the database (as an SQL stored procedure); it's implemented in Java and executed in the application tier. This allows the business logic to use sophisticated object-oriented concepts such as inheritance and polymorphism. For example, we could use well-known design patterns such as *strategy*, *mediator*, and *composite* (see *Design Patterns: Elements of Reusable Object-Oriented Software* [Gamma, 1994]), all of which depend on polymorphic method calls.

Now a warning: not all Java applications are designed this way, nor should they be. Simple applications may be much better off without a domain model. Use the JDBC `ResultSet` if that's all you need. Call existing stored procedures, and read their SQL result sets, too. Many applications need to execute procedures that modify large sets of data, close to the data. You might also implement some reporting functionality with plain SQL queries and render the results directly onscreen. SQL and the JDBC API are perfectly serviceable for dealing with tabular data representations, and the JDBC `RowSet` makes CRUD operations even easier. Working with such a representation of persistent data is straightforward and well understood.

But for applications with nontrivial business logic, the domain model approach helps to improve code reuse and maintainability significantly. In practice, *both* strategies are common and needed.

For several decades, developers have spoken of a *paradigm mismatch*. The *paradigms* referred to are object modeling and relational modeling, or, more practically, object-oriented programming and SQL. This mismatch explains why every enterprise project expends so much effort on persistence-related concerns. With this conception, you can begin to see the problems—some well understood and some less well understood—that must be solved in an application that combines an object-oriented domain model and a persistent relational model. Let’s take a closer look at this so-called paradigm mismatch.

1.2 The paradigm mismatch

The object/relational paradigm mismatch can be broken into several parts, which we’ll examine one at a time. Let’s start our exploration with a simple example that is problem-free. As we build on it, you’ll see the mismatch begin to appear.

Suppose you have to design and implement an online e-commerce application. In this application, you need a class to represent information about a user of the system, and you need another class to represent information about the user’s billing details, as shown in figure 1.1.

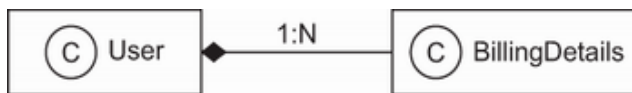


Figure 1.1 A simple UML diagram of the `User` and `BillingDetails` entities

In this diagram, you can see that a `User` has many `BillingDetails`. This is a composition, indicated by the full diamond. A composition is the type of association where an object (`BillingDetails` in our case) cannot conceptually exist without the container (`User` in our case). You can navigate the relationship between the classes in both directions; this means you can iterate through collections or call methods to get to the “other” side of the relationship. The classes representing these entities may be extremely simple:

```
Path: Ch01/e-commerce/src/com/manning/javapersistence/ch01/User.java
```

```
public class User {
    private String username;
    private String address;
    private Set<BillingDetails> billingDetails = new HashSet<>();

    // Constructor, accessor methods (getters/setters), business met
```

```
}
```

```
Path: Ch01/e-commerce/src/com/manning/javapersistence/ch01
```

```
➡ /BillingDetails.java
```

```
public class BillingDetails {  
    private String account;  
    private String bankname;  
    private User user;  
  
    // Constructor, accessor methods (getters/setters), business met  
}
```

Note that we're only interested in the state of the entities' persistence, so we've omitted the implementation of constructors, accessor methods, and business methods.

It's easy to come up with an SQL schema design for this case (the syntax of the following queries is applicable to MySQL):

```
CREATE TABLE USERS (  
    USERNAME VARCHAR(15) NOT NULL PRIMARY KEY,  
    ADDRESS VARCHAR(255) NOT NULL  
);  
  
CREATE TABLE BILLINGDETAILS (  
    ACCOUNT VARCHAR(15) NOT NULL PRIMARY KEY,  
    BANKNAME VARCHAR(255) NOT NULL,  
    USERNAME VARCHAR(15) NOT NULL,  
    FOREIGN KEY (USERNAME) REFERENCES USERS(USERNAME)  
);
```

The foreign key–constrained column `USERNAME` in `BILLINGDETAILS` represents the relationship between the two entities. For this simple domain model, the object/ relational mismatch is barely in evidence; it's straightforward to write JDBC code to insert, update, and delete information about users and billing details.

Now let's see what happens when we consider something a little more realistic. The paradigm mismatch will be visible when we add more entities and entity relationships to the application.

1.2.1 The problem of granularity

The most obvious problem with the current implementation is that we've designed an address as a simple `String` value. In most systems, it's nec-

essary to store street, city, state, country, and ZIP code information separately. Of course, you could add these properties directly to the `User` class, but because other classes in the system will likely also carry address information, it makes more sense to create an `Address` class to reuse it. Figure 1.2 shows the updated model.

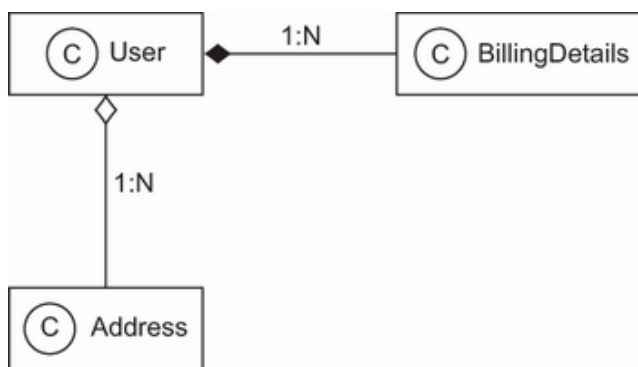


Figure 1.2 The `User` has an `Address`.

The relationship between `User` and `Address` is an aggregation, indicated by the empty diamond. Should we also add an `ADDRESS` table? Not necessarily; it's common to keep address information in the `USERS` table, in individual columns. This design is likely to perform better because a table join isn't needed if you want to retrieve the user and address in a single query. The nicest solution may be to create a new SQL data type to represent addresses and to add a single column of that new type in the `USERS` table, instead of adding several new columns.

This choice of adding either several columns or a single column of a new SQL data type is a problem of *granularity*. Broadly speaking, granularity refers to the relative size of the types you're working with.

Let's return to the example. Adding a new data type to the database catalog to store `Address` Java instances in a single column sounds like the best approach:

```
CREATE TABLE USERS (  
    USERNAME VARCHAR(15) NOT NULL PRIMARY KEY,  
    ADDRESS ADDRESS NOT NULL  
);
```

A new `Address` type (class) in Java and a new `ADDRESS` SQL data type should guarantee interoperability. But you'll find various problems if you check on the support for user-defined data types (UDTs) in today's SQL database management systems.

UDT support is one of several so-called *object/relational extensions* to traditional SQL. This term alone is confusing, because it means the database management system has (or is supposed to support) a sophisticated data type system. Unfortunately, UDT support is a somewhat obscure feature of most SQL DBMSs, and it certainly isn't portable between different products. Furthermore, the SQL standard supports user-defined data types, but poorly.

This limitation isn't the fault of the relational data model. You can consider the failure to standardize such an important piece of functionality to be a result of the object/relational database wars between vendors in the mid-1990s. Today most engineers accept that SQL products have limited type systems—no questions asked. Even with a sophisticated UDT system in your SQL DBMS, you would still likely duplicate the type declarations, writing the new type in Java and again in SQL. Attempts to find a better solution for the Java space, such as SQLJ, unfortunately have not had much success. DBMS products rarely support deploying and executing Java classes directly on the database, and if support is available, it's typically limited to very basic functionality in everyday usage.

For these and whatever other reasons, the use of UDTs or Java types in an SQL database isn't common practice at this time, and it's unlikely that you'll encounter a legacy schema that makes extensive use of UDTs. We therefore can't and won't store instances of our new `Address` class in a single new column that has the same data type as the Java layer.

The pragmatic solution for this problem has several columns of built-in vendor-defined SQL types (such as Boolean, numeric, and string data types). You'd usually define the `USERS` table as follows:

```
CREATE TABLE USERS (  
    USERNAME VARCHAR(15) NOT NULL PRIMARY KEY,  
    ADDRESS_STREET VARCHAR(255) NOT NULL,  
    ADDRESS_ZIPCODE VARCHAR(5) NOT NULL,  
    ADDRESS_CITY VARCHAR(255) NOT NULL  
);
```

Classes in the Java domain model come in a range of levels of granularity: from coarse-grained entity classes like `User` to finer-grained classes like `Address`, down to simple `SwissZipCode` extending `AbstractNumericZipCode` (or whatever your desired level of abstraction is). In contrast, just two levels of type granularity are visible in the SQL database: relation types created by you, like `USERS` and

`BILLINGDETAILS`, and built-in data types such as `VARCHAR`, `BIGINT`, and `TIMESTAMP`.

Many simple persistence mechanisms fail to recognize this mismatch and so end up forcing the less flexible representation of SQL products on the object-oriented model, effectively flattening it. It turns out that the granularity problem isn't especially difficult to solve, even if it's visible in so many existing systems. We'll look at the solution to this problem in section 5.1.1.

A much more difficult and interesting problem arises when we consider domain models that rely on *inheritance*, a feature of object-oriented design you may use to bill the users of your e-commerce application in new and interesting ways.

1.2.2 The problem of inheritance

In Java, you implement type inheritance using superclasses and subclasses. To illustrate why this can present a mismatch problem, let's modify our e-commerce application so that we now can accept not only bank account billing, but also credit cards. The most natural way to reflect this change in the model is to use inheritance for the `BillingDetails` superclass, along with multiple concrete subclasses: `CreditCard`, `BankAccount`. Each of these subclasses defines slightly different data (and completely different functionality that acts on that data). The UML class diagram in figure 1.3 illustrates this model.

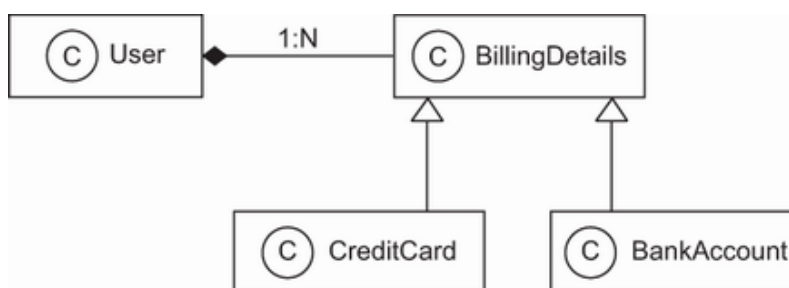


Figure 1.3 Using inheritance for different billing strategies

What changes must we make to support this updated Java class structure? Can we create a `CREDITCARD` table that *extends* `BILLINGDETAILS`? SQL database products don't generally implement table inheritance (or even data type inheritance), and if they do implement it, they don't follow a standard syntax.

We haven't finished with inheritance. As soon as we introduce inheritance into the model, we have the possibility of *polymorphism*. The `User`

class has a *polymorphic association* with the `BillingDetails` superclass. At runtime, a `User` instance may reference an instance of any of the subclasses of `BillingDetails`. Similarly, we want to be able to write *polymorphic queries* that refer to the `BillingDetails` class and have the query return instances of its subclasses.

SQL databases lack an obvious way (or at least a standardized way) to represent a polymorphic association. A foreign key constraint refers to exactly one target table; it isn't straightforward to define a foreign key that refers to multiple tables.

The result of this mismatch of subtypes is that the inheritance structure in a model must be persisted in an SQL database that doesn't offer an inheritance mechanism. In chapter 7 we'll discuss how ORM solutions such as Hibernate solve the problem of persisting a class hierarchy to an SQL database table or tables, and how polymorphic behavior can be implemented. Fortunately, this problem is now well understood in the community, and most solutions support approximately the same functionality.

The next aspect of the object/relational mismatch problem is the issue of *object identity*.

1.2.3 The problem of identity

You probably noticed that the example defined `USERNAME` as the primary key of the `USERS` table. Was that a good choice? How do you handle identical objects in Java?

Although the problem of identity may not be obvious at first, you'll encounter it often in your growing and expanding e-commerce system, such as when you need to check whether two instances are identical. There are three ways to tackle this problem: two in the Java world and one in the SQL database. As expected, they work together only with some help.

Java defines two different notions of *sameness*:

- Instance identity (roughly equivalent to a memory location, checked with `a == b`)
- Instance equality, as determined by the implementation of the `equals()` method (also called *equality by value*)

On the other hand, the identity of a database row is expressed as a comparison of primary key values. As you'll see in section 9.1.2, neither `equals()` nor `==` is always equivalent to a comparison of primary key values. It's common for several non-identical instances in Java to simulta-

neously represent the same row of a database, such as in concurrently running application threads. Furthermore, some subtle difficulties are involved in implementing `equals()` correctly for a persistent class and in understanding when this might be necessary.

Let's use an example to discuss another problem related to database identity. In the table definition for `USERS`, `USERNAME` is the primary key. Unfortunately, this decision makes it difficult to change a user's name; you need to update not only the row in `USERS` but also the foreign key values in (many) rows of `BILLINGDETAILS`. To solve this problem, later in this book we'll recommend that you use *surrogate keys* whenever you can't find a good natural key. We'll also discuss what makes a good primary key. A surrogate key column is a primary key column with no meaning to the application user—in other words, a key that isn't presented to the application user. Its only purpose is to identify data inside the application.

For example, you may change your table definitions to look like this:

```
CREATE TABLE USERS (  
    ID BIGINT NOT NULL PRIMARY KEY,  
    USERNAME VARCHAR(15) NOT NULL UNIQUE,  
    . . .  
);  
CREATE TABLE BILLINGDETAILS (  
    ID BIGINT NOT NULL PRIMARY KEY,  
    ACCOUNT VARCHAR(15) NOT NULL,  
    BANKNAME VARCHAR(255) NOT NULL,  
    USER_ID BIGINT NOT NULL,  
    FOREIGN KEY (USER_ID) REFERENCES USERS(ID)  
);
```

The `ID` columns contain system-generated values. These columns were introduced purely for the benefit of the data model, so how (if at all) should they be represented in the Java domain model? We'll discuss this question in section 5.2, and we'll find a solution with ORM.

In the context of persistence, identity is closely related to how the system handles caching and transactions. Different persistence solutions have chosen different strategies, and this has been an area of confusion. We'll cover all these interesting topics—and look at how they're related—in section 9.1.

So far, the skeleton e-commerce application we've designed has exposed the paradigm mismatch problems with mapping granularity, subtypes,

and identity. We need to discuss further the important concept of *associations*: how the relationships between entities are mapped and handled. Is the foreign key constraint in the database all you need?

1.2.4 The problem of associations

In the domain model, associations represent the relationships between entities. The `User`, `Address`, and `BillingDetails` classes are all associated; but unlike `Address`, `BillingDetails` stands on its own.

`BillingDetails` instances are stored in their own table. Association mapping and the management of entity associations are central concepts in any object persistence solution.

Object-oriented languages represent associations using *object references*, but in the relational world, a *foreign key–constrained column* represents an association with copies of key values. The constraint is a rule that guarantees the integrity of the association. There are substantial differences between the two mechanisms.

Object references are inherently directional; the association is from one instance to the other. They're pointers. If an association between instances should be navigable in both directions, you must define the association *twice*, once in each of the associated classes. The UML class diagram in figure 1.4 illustrates this model with a one-to-many association.

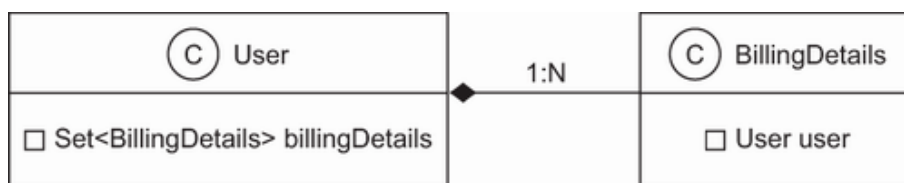


Figure 1.4 One-to-many association between `User` and `BillingDetails`

You've already seen this in the domain model classes:

```
Path: Ch01/e-commerce/src/com/manning/javapersistence/ch01/User.java
```

```
public class User {
    private Set<BillingDetails> billingDetails = new HashSet<>();
}
```

```
Path: Ch01/e-commerce/src/com/manning/javapersistence/ch01
➡ /BillingDetails.java
```

```
public class BillingDetails {
```



```

        private User user;
    }

```

Navigation in a particular direction has no meaning for a relational data model because you can create data associations with *join* and *projection* operators. The challenge is to map a completely open data model that is independent of the application that works with the data to an application-dependent navigational model—a constrained view of the associations needed by this particular application.

Java associations can have *many-to-many* multiplicity. The UML class diagram in figure 1.5 illustrates this model.

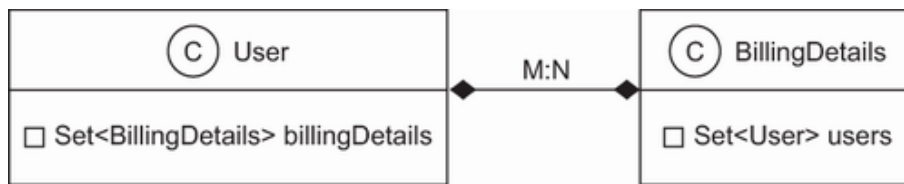


Figure 1.5 Many-to-many association between User and BillingDetails

The classes could look like this:

Path: Ch01/e-commerce/src/com/manning/javapersistence/ch01/User.java

```

public class User {
    private Set<BillingDetails> billingDetails = new HashSet<>();
}

```

Path: Ch01/e-commerce/src/com/manning/javapersistence/ch01
 ➡ /BillingDetails.java

```

public class BillingDetails {
    private Set<User> users = new HashSet<>();
}

```

However, the foreign key declaration on the `BILLINGDETAILS` table is a *many-to-one* association: each bank account is linked to a particular user, but each user may have multiple linked bank accounts.

If you wish to represent a *many-to-many* association in an SQL database, you must introduce a new table, usually called a *link table*. In most cases, this table doesn't appear anywhere in the domain model. For this example, if you consider the relationship between the user and the billing information to be many-to-many, you would define the link table as follows:

```
CREATE TABLE USER_BILLINGDETAILS (
    USER_ID BIGINT,
    BILLINGDETAILS_ID BIGINT,
    PRIMARY KEY (USER_ID, BILLINGDETAILS_ID),
    FOREIGN KEY (USER_ID) REFERENCES USERS(ID),
    FOREIGN KEY (BILLINGDETAILS_ID) REFERENCES BILLINGDETAILS(ID)
);
```

You no longer need the `USER_ID` foreign key column and constraint on the `BILLINGDETAILS` table; this additional table now manages the links between the two entities. We'll discuss association and collection mappings in detail in chapter 8.

So far, the problems we've considered are mainly *structural*: you can see them by considering a purely static view of the system. Perhaps the most difficult problem in object persistence is a *dynamic* problem: how data is accessed at runtime.

1.2.5 The problem of data navigation

There is a fundamental difference between how you access data in Java code and within a relational database. In Java, when you access a user's billing information, you call `someUser.getBillingDetails().iterator().next()` or something similar. Or, starting from Java 8, you may call `someUser.getBillingDetails().stream().filter(someCondition).map(someMapping).forEach(billingDetails-> {doSomething (billingDetails)})`. This is the most natural way to access object-oriented data, and it's often described as *walking the object network*. You navigate from one instance to another, even iterating collections, following prepared pointers between classes. Unfortunately, this isn't an efficient way to retrieve data from an SQL database.

The single most important thing you can do to improve the performance of data access code is to *minimize the number of requests to the database*. The most obvious way to do this is to minimize the number of SQL queries. (Of course, other, more sophisticated, ways—such as extensive caching—follow as a second step.)

Therefore, efficient access to relational data with SQL usually requires joins between the tables of interest. The number of tables included in the join when retrieving data determines the depth of the object network you can navigate in memory. For example, if you need to retrieve a `User` and aren't interested in the user's billing information, you can write this simple query:

```
SELECT * FROM USERS WHERE ID = 123
```

On the other hand, if you need to retrieve a `User` and then subsequently visit each of the associated `BillingDetails` instances (let's say, to list the user's bank accounts), you would write a different query:

```
SELECT * FROM USERS, BILLINGDETAILS  
WHERE USERS.ID = 123 AND  
BILLINGDETAILS.ID = USERS.ID
```

As you can see, to use joins efficiently you need to know what portion of the object network you plan to access *before* you start navigating the object network! Careful, though: if you retrieve too much data (probably more than you might need), you're wasting memory in the application tier. You may also overwhelm the SQL database with huge Cartesian product result sets. Imagine retrieving not only users and bank accounts in one query, but also all orders paid from each bank account, the products in each order, and so on.

Any object persistence solution permits you to fetch the data of associated instances only when the association is first accessed in the Java code. This is known as *lazy loading*: retrieving data only on demand. This piecemeal style of data access is fundamentally inefficient in the context of an SQL database, because it requires executing one statement for each node or collection of the object network that is accessed. This is the dreaded *n+1 selects* problem. In our example, you will need one *select* to retrieve a `User` and then *n selects* for each of the *n* associated `BillingDetails` instances.

This mismatch in the way you access data in Java code and within a relational database is perhaps the single most common source of performance problems in Java information systems. Avoiding the Cartesian product and *n+1 selects* problems is still a problem for many Java programmers. Hibernate provides sophisticated features for efficiently and transparently fetching networks of objects from the database to the application accessing them. We'll discuss these features in chapter 12.

We now have quite a list of object/relational mismatch problems: the problem of granularity, the problem of inheritance, the problem of identity, the problem of associations, and the problem of data navigation. It can be costly (in time and effort) to find solutions, as you may know from experience. It will take us a large part of this book to provide detailed an-

swers to these questions and to demonstrate ORM as a viable solution. Let's get started with an overview of ORM, the Java Persistence standard (JPA), and the Hibernate and Spring Data projects.

1.3 ORM, JPA, Hibernate, and Spring Data

In a nutshell, object/relational mapping (ORM) is the automated (and transparent) persistence of objects in a Java application to the tables in an RDBMS (relational database management system), using metadata that describes the mapping between the classes of the application and the schema of the SQL database. In essence, ORM works by transforming (reversibly) data from one representation to another. A program using ORM will provide the meta-information about how to map the objects from the memory to the database, and the effective transformation will be fulfilled by ORM.

Some people may consider one advantage of ORM to be that it shields developers from messy SQL. This view holds that object-oriented developers shouldn't be expected to go deep into SQL or relational databases. On the contrary, Java developers must have a sufficient level of familiarity with—and appreciation of—relational modeling and SQL to work with Hibernate and Spring Data. ORM is an advanced technique used by developers who have already done it the hard way.

JPA (Jakarta Persistence API, formerly Java Persistence API) is a specification defining an API that manages the persistence of objects and object/relational mappings. Hibernate is the most popular implementation of this specification. So, JPA will specify what must be done to persist objects, while Hibernate will determine how to do it. Spring Data Commons, as part of the Spring Data family, provides the core Spring framework concepts that support all Spring Data modules. Spring Data JPA, another project from the Spring Data family, is an additional layer on top of JPA implementations (such as Hibernate). Not only can Spring Data JPA use all the capabilities of JPA, but it adds its own capabilities, such as generating database queries from method names. We'll go into many details in this book, but if you would like an overall view right now, you can quickly jump ahead to figure 4.1.

To use Hibernate effectively, you must be able to view and interpret the SQL statements it issues and understand their performance implications. To take advantage of the benefits of Spring Data, you must be able to anticipate how the boilerplate code and the generated queries are created.

The JPA specification defines the following:

- A facility for specifying mapping metadata—how persistent classes and their properties relate to the database schema. JPA relies heavily on Java annotations in domain model classes, but you can also write mappings in XML files.
- APIs for performing basic CRUD operations on instances of persistent classes, most prominently `javax.persistence.EntityManager` for storing and loading data.
- A language and APIs for specifying queries that refer to classes and properties of classes. This language is the Jakarta Persistence Query Language (JPQL) and it looks similar to SQL. The standardized API allows for the programmatic creation of *criteria queries* without string manipulation.
- How the persistence engine interacts with transactional instances to perform dirty checking, association fetching, and other optimization functions. The JPA specification covers some basic caching strategies.

Hibernate implements JPA and supports all the standardized mappings, queries, and programming interfaces. Let's look at some of the benefits of Hibernate:

- *Productivity*—Hibernate eliminates much of the repetitive work (more than you'd expect) and lets you concentrate on the business problem. No matter which application-development strategy you prefer—top-down (starting with a domain model) or bottom-up (starting with an existing database schema)—Hibernate, used together with the appropriate tools, will significantly reduce development time.
- *Maintainability*—Automated ORM with Hibernate reduces lines of code, making the system more understandable and easier to refactor. Hibernate provides a buffer between the domain model and the SQL schema, isolating each model from minor changes to the other.
- *Performance*—Although hand-coded persistence might be faster in the same sense that assembly code can be faster than Java code, automated solutions like Hibernate allow the use of many optimizations *at all times*. One example is the efficient and easily tunable caching in the application tier. This means developers can spend more energy hand-optimizing the few remaining real bottlenecks instead of prematurely optimizing everything.
- *Vendor independence*—Hibernate can help mitigate some of the risks associated with vendor lock-in. Even if you plan never to change your DBMS product, ORM tools that support several different DBMSs enable a certain level of portability. Also, DBMS independence helps in development scenarios where engineers use a lightweight local database but deploy for testing and production on a different system.

Spring Data makes the implementation of the persistence layer even more efficient. Spring Data JPA, one of the projects of the family, sits on top of the JPA layer. Spring Data JDBC, another project of the family, sits on top of JDBC. Let's look at some of the benefits of Spring Data:

- *Shared infrastructure*—Spring Data Commons, part of the umbrella Spring Data project, provides a metadata model for persisting Java classes and technology-neutral repository interfaces. It provides its capabilities to the other Spring Data projects.
- *Removes DAO implementations*—JPA implementations use the *data access object* (DAO) pattern. This pattern starts with the idea of an abstract interface to a database and maps application calls to the persistence layer while hiding the details of the database. Spring Data JPA makes it possible to fully remove DAO implementations, so the code will be shorter.
- *Automatic class creation*—Using Spring Data JPA, a DAO interface needs to extend the JPA-specific `Repository` interface—`JpaRepository`. Spring Data JPA will automatically create an implementation for this interface—the programmer will not have to take care of this.
- *Default implementations for methods*—Spring Data JPA will generate default implementations for each method defined by its repository interfaces. Basic CRUD operations do not need to be implemented any longer. This reduces the boilerplate code, speeds up development, and removes the possibility of introducing bugs.
- *Generated queries*—You may define a method on your repository interface following a naming pattern. There's no need to write your queries by hand; Spring Data JPA will parse the method name and create a query for it.
- *Close to the database if needed*—Spring Data JDBC can communicate directly with the database and avoid the “magic” of Spring Data JPA. It allows you to interact with the database through JDBC, but it removes the boilerplate code by using the Spring framework facilities.

This chapter has focused on understanding object/relational persistence and the problems generated by the object/relational paradigm mismatch. Chapter 2 will look at some of the persistence alternatives for a Java application: JPA, Hibernate Native, and Spring Data JPA.

Summary

- With *object persistence*, individual objects can outlive their application process, be saved to a data store, and be re-created later. The object/relational mismatch comes into play when the data store is an SQL-based relational database management system. For example, a network of objects can't be saved to a database table; it must be disassembled and persisted to columns of portable SQL data types. A good solution to this problem is object/relational mapping (ORM).
- ORM isn't a silver bullet for all persistence tasks; its job is to relieve the developer of about 95% of object persistence work, such as writing complex SQL statements with many table joins and copying values from JDBC result sets to objects or graphs of objects.
- A full-featured ORM middleware solution may provide database portability, certain optimization techniques like caching, and other viable functions that aren't easy to hand-code in a limited time with SQL and JDBC. An ORM solution implies, in the Java world, the JPA specification and a JPA implementation—Hibernate being the most popular nowadays.
- Spring Data may come on top of the JPA implementations, and it simplifies, even more, the data persistence process. It is an umbrella project that adheres to the Spring framework principles and comes with an even simpler approach, including removing the DAO pattern, automatic code generation, and automatic query generation.